

計算機概論期末專題

第十二組

B812111001 張博翔

B812111029 陳南方

1. 程式碼：

```
#include <stdio.h>
#include <time.h>
#include <stdlib.h>

void around(int a[], int b, int c[], int d, int e, int f, int g, int z);
void printans(int a[], int b);
void printmap(int a[], int b);

int main(){
    double record[1000]; //歷史紀錄的資料庫
    int regame = 0;      //紀錄已進行幾次遊戲
    int game = 0;        //輸入面板的變數
    int maingame = 1;    //開始遊戲的變數

    for(int i = 0; i < 1000; i++){
        record[i] = -1;
    }

    while(maingame == 1){
        //遊戲面板
        printf("◆◆◆◆◆◆◆◆◆◆◆◆◆◆◆◆\n\n");
        printf("-----*開始遊戲*-----\n");
        printf("-----*請輸入:1*-----\n\n");
        printf("-----#結束遊戲#-----\n");
        printf("-----#請輸入:0#-----\n\n");
        printf("-----§查看遊戲紀錄§-----\n");
        printf("-----§請輸入:2§-----\n\n");
        printf("◆◆◆◆◆◆◆◆◆◆◆◆◆◆◆◆\n");
    }
}
```

```
printf("輸入:");
scanf("%d",&game);

//輸入不在面板上，結束遊戲
if(game !=1 && game !=2){
    maingame = 0;
}

//用於顯示歷史紀錄
int printrecord = 0;

if(game == 2){
    system("cls");
    printf("◎ 紀錄:\n");
}

while(game == 2){

    if(record[printrecord] != -1){
        printf("    %5.11f [是否勝利: %2.01f , 花費時間: %7.21fsec , 使用步數: %4.01f 步]\n", record[printrecord], record[printrecord+1], record[printrecord+2], record[printrecord+3]);
        printrecord = printrecord+4;
    }

    else{
        printf("\n 輸入 1 開始遊戲, 0 回到主選單");
        scanf("%d",&game);
        system("cls");
    }
}

//進入踩地雷遊戲
while(game == 1){
    regame++;

    double begin = 0; //開始時間
    double end = 0;   //結束時間
```

```
int step = 0;    //使用步數
int size = 0;    //設定地圖大小的變數
int n = 99999;   //設定地雷數量的變數

//輸入地圖大小
system("cls");
printf("輸入範圍(r*r) :");
scanf("%d",&size);

//輸入地雷數量
while(n > size*size-1){
    printf("炸彈數量 :");
    scanf("%d",&n);

    if(n >= size*size){
        printf("!!!【請重新輸入】!!!\n");
    }
}

//設定串列的原始數值
int a[size*size];
int c = 0;
for(int i = 0; i < size*size; i++){
    a[i] = 0 ;
}

//在地圖中放地雷
while(c < n){
    //取隨機數
    srand(time(NULL)*(c+1));
    int pos = rand()%(size*size);
    around(a, size, a, 1, 1000, 1, 10, pos);
    c++;
}

//設定初始地圖
int amap[size*size];
for(int i = 0; i < size*size; i++){
```

```
        amap[i] = 99;
    }

    //遊戲開始前先顯示地圖
    printmap(amap,size);

    //開始遊戲
    int start = 1;
    int t = 0;
    begin = time(NULL); //記錄遊戲開始時間

    while(start == 1){
        //輸入操作的三個數值
        int x = -1;
        int y = -1;
        int function = 0;

        //輸入 X 座標
        while(x >= size || x < 0){ //迴圈用於確定輸入範圍正確
            printf("\nX: ");
            scanf("%d",&x);
        }

        //輸入 Y 座標
        while(y >= size || y < 0){
            printf("\nY: ");
            scanf("%d",&y);
        }

        //輸入要執行的功能（1 是翻開，2 是標記，3 是作弊：顯示解答）
        while(function !=1 && function !=2 && function !=3){
            printf("\nFunction: ");
            scanf("%d",&function);
        }
        step++; //完成一次輸入則步數+1

        //執行功能：顯示解答
        if(function == 3){
```

```

        printans(a,size);
    }

    //執行功能：標記
    else if(function == 2){
        if(amap[x+y*size] < 100){
            amap[x+y*size] = 100;
        }

        //執行功能：取消標記
        else{
            amap[x+y*size] = 99;
        }

        //標記後顯示地圖
        printmap(amap,size);

        //顯示需要翻開的格子數以及已翻開的格子數
        printf("\n  需翻開:%d , 已翻開:%d\n", size*size-n, t);
    }

    else{
        //失敗條件：踩到炸彈
        if(a[x+y*size] >= 1000){
            printf("! ★ ☆ BOOM ☆ ★ !");
            printf("\n\n\map:\n");
            printf("\n ");
            record[regame*4-3] = 0;

            //失敗後顯示地圖
            printans(a,size);
            start = 0;
        }

        //勝利條件：翻開所有不是炸彈的格子
        else if(size*size-n <= t+1){
            printf("! ★ ☆ 勝利 ☆ ★ !\n");
            record[regame*4-3] = 1;
        }
    }
}

```

```

        //勝利後顯示地圖
        printans(a,size);
        start = 0;
    }

    //執行功能：翻開
    else{
        amap[x+y*size] = a[x+y*size];

        //加分項目：翻開周圍的格子
        around(amap, size, a, 9999999, -10, 0, 0, 0); //9999999 為限制運算次數用

        int check = 0;
        while(check < size*size){
            if(amap[check] == 0){
                amap[check] = -10;
            }
            check++;
        }

        //計算已翻開格子數
        t = 0;
        for(int i = 0; i<size*size; i++){
            if(amap[i] < 50){
                t++;
            }
        }

        //顯示目前的地圖
        printmap(amap,size);

        printf("\n  需翻開:%d , 已翻開:%d\n", size*size-n, t);
    }
}
}

```

```

        end = time(NULL); //紀錄遊戲結束時間

        //遊戲結束
        printf("\n【遊戲結束】!\n 時間為: %.3lf sec\n",end - begin);
        record[regame*4-1] = step;
        record[regame*4-2] = end - begin;
        record[regame*4-4] = regame;
        printf("\n1 : 重新遊戲\n0 : 返回選單\n");
        scanf("%d",&game);
    }
}
return 0;
}

void around(int a[],int b,int c[],int d,int e,int f,int g,int z){ //處理地圖邊緣特殊狀況
    int y = 0;

    while(z < b*b && y < d){
        if(a[z] <= g && a[z]>=0){
            if(z % b != b - 1){
                a[z+1] = c[z+1]+f;
            }
            if(z % b != 0){
                a[z-1] = c[z-1]+f;
            }
            if(z / b != b - 1){
                a[z+b] = c[z+b]+f;
            }
            if(z / b != 0){
                a[z-b] = c[z-b]+f;
            }
            if(z / b != b - 1 && z % b != b - 1){
                a[z+b+1] = c[z+b+1]+f;
            }
            if(z / b != b - 1 && z % b != 0){
                a[z+b-1] = c[z+b-1]+f;
            }
        }
    }
}

```

```

        if(z / b != 0 && z % b != 0){
            a[z-b-1] = c[z-b-1]+f;
        }
        if(z / b != 0 && z % b != b - 1){
            a[z-b+1] = c[z-b+1]+f;
        }
        a[z] = e;
        z = 0;
        y++;
    }

    else{
        z++;
    }
}

}

void printans(int a[],int b){ //顯示解答
    printf("    ");
    for(int i = 0; i < b; i++){
        printf("(%2d)",i);
    }

    for(int j = 0;j < b;j++){
        printf("\n(%2d)",j);

        for(int i = 0;i <b;i++){
            if(a[i+j*b] >= 1000){
                printf(" \033[41;5m* \033[0m");
            }

            else{
                printf(" \033[42;5m%2d \033[0m",a[i+j*b]);
            }
        }
    }
}
}

```

```

void printmap(int a[],int b){ //顯示地圖
    printf("\n ");

    for(int i = 0;i <b;i++){
        printf(" %2d",i);
    }

    for(int j = 0;j <b;j++){
        printf("\n%2d",j);

        for(int i = 0;i <b;i++){
            if(a[i+j*b] == -10){
                printf(" \033[42;5m%d \033[0m",0);
            }
            else if(a[i+j*b] == 100){
                printf(" P ");
            }
            else if(a[i+j*b] > 50){
                printf(" ■ ");
            }
            else{
                printf(" \033[42;5m%d \033[0m",a[i+j*b]);
            }

        }
    }
}

```

• 程式碼 Demo：

https://drive.google.com/file/d/1PTEUkt58wD8T8YSxh_mHBkWo0cPMqri/view?usp=sharing

2. 已完成的加分項目：

- 自訂遊戲範圍與地雷數
- 若開啟一個周圍都沒地雷的位置，會開啟整片區域直到遇到周圍存在地雷

- 儲存及查看歷史遊玩紀錄（是否勝利、時間、步數）
- 顯示遊玩時間
- 美化程式，輸出的數字與地雷背景皆有顏色
- 查看地雷位置的功能

3. 流程圖：

<https://drive.google.com/file/d/1rh-tEnVufmewAzucCzGT5orbhUMC5dZw/view?usp=sharing>
點進雲端可以觀看，不然會太多張

4. 組員心得：

張博翔：

期末專題與期中時相比根本天差地別，除錯佔了大部分時間，其實我們本來有另一個版本是試著用自定義去做，不過因為遲遲無法改好，所以才寫出這一版本。在撰寫時遇到了許多的問題，由於系統問題，程式中如 `system("cls")` 他可能就無法執行，所以除錯時都是以我分享螢幕給他看一起修改，而資訊上因為兩個人每次寫出來的東西會因此不一樣而在整合上花了許多時間。這一次專題討論真的是非常的重要，像是執行程式碼時常會遇到一些 bug 是你可能無法一人輕鬆解決的，如後續附加的判別式處理周圍特殊狀況兩人討論就花了不少的時間。我認為我們這組程式碼有太多地方需要改進，如翻開周圍無地雷的區域因為重複程式所以會發現顯示速度變慢，或者是處理地雷放置的程式碼，光是列出了特殊條件就有八項，所以變得非常冗長。不過當中也有幾項我覺得處理得很好的就是遊戲循環方面，已經將所有可以返回的地方都設置好了迴圈，不會遇到不斷重開的問題也方便了許多。透過這次的報告也讓我知道自己還有很大的進步空間，前方還有很長的一段路要走，雖然可以改進的地方仍有很多，不過完成的時候感到的成就感卻不會少，也讓我更加知道自己需要更努力。

陳南方：

因為我是用 apple 電腦，所以我有一些功能沒辦法用，美化的東西就全部交給組員。過程中遇到兩個比較嚴重的問題。一個是因為我們用的是一維的陣列，所以在邊緣時可能會出現旁邊明明沒有格子卻誤判成下一行的格子的情況，為此我寫了 8 個暴力解決問題的 if，用來處理邊緣的特殊情況。另一個是開啟一整片區域的功能遇到的問題，當地圖中有一大塊沒有地雷的區域時，開啟一整片區域的程式碼會運算很多次導致遊戲變很慢，雖然在地圖不大或地雷夠多的情況下不會發生問題，但因為我們也有做自訂地圖大小和地雷數的功能，所以就很容易出問題。因此我不得不給它加上運算次數的上限，雖然還是會有點卡但已經好很多了。而為了解決每次開啟區域邊緣地區會被這段程式反覆開啟導致遊戲變慢的問題，我在每次開啟一片區域的程式結束後都會改變被開啟的格子的值，使她依然顯示 0 但無法再次被開啟一片區域的程式開啟。

雖然經過以上處理後這段程式的效率有變好，但還是會有運行的上限。不過目前的狀況可以確定在 10×10 ，15 個地雷的地圖幾乎不會有問題（除非地雷全都擠在地圖的邊緣）。

5. 分工表：

- 程式碼撰寫：張博翔、陳南方
- 流程圖：張博翔、陳南方
- 影片錄製：張博翔